

AdaIN StyleKV: Training-Free Style Transfer with Self-Attention KV Fusion and Adaptive Instance Normalization for Diffusion Models

Junji Jiang Zhuoliang Lv Peiwen Yang

School of Computer Science and Engineering (Cyberspace Security),
University of Electronic Science and Technology of China

{junji.jiang, zhuoliang.lv, peiwen.yang}@uestc.edu.cn

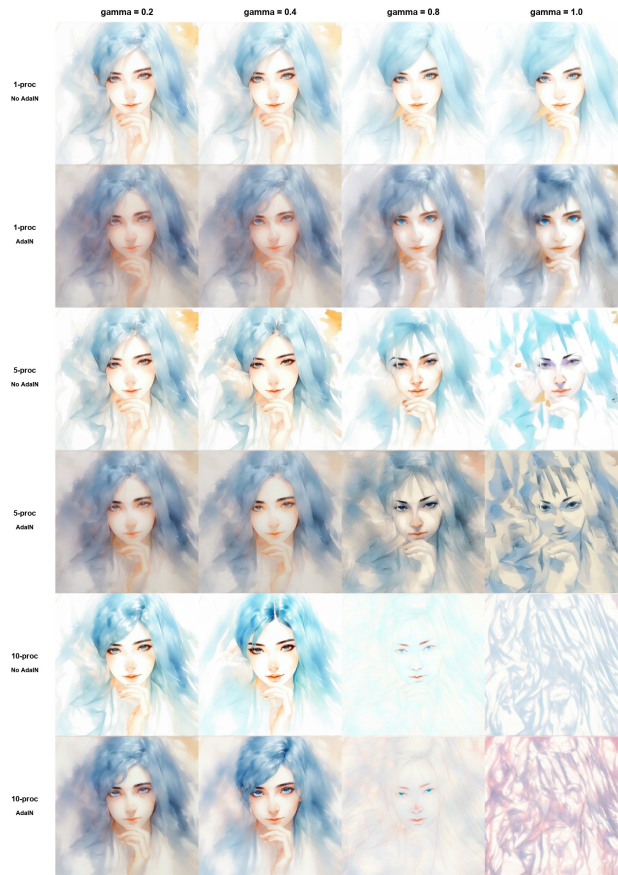


Fig. 1. Stylization results of AdaIN StyleKV. Horizontally, γ increases from 0.2 to 1.0; vertically, 1-proc, 5-proc, and 10-proc configurations are shown with and without AdaIN. The 5-proc + AdaIN configuration (our full method) gives a useful balance between style intensity and content fidelity near $\gamma = 0.6$.

Abstract

Artistic style transfer applies the appearance of a reference image to a content image while preserving the scene layout. Diffusion models have improved visual quality, but current methods still make trade-offs. Some approaches require per-style fine-tuning or inversion. Training-free methods avoid this cost, but they can drift in structure, miss global color statistics, or offer limited control over style intensity. We propose AdaIN StyleKV, a training-free framework built on a frozen diffusion model. The method combines Canny ControlNet for structural guidance, selective self-attention K/V fusion for texture transfer, and a cross-modal AdaIN module adapted from StyleStudio [31] for color and condition alignment. In the target UNet block `up_blocks.0.attentions.1`, we cache K/V features from the style image and blend them with the content K/V using a scalar coefficient γ . At the same block, AdaIN uses channel statistics from a pretrained image embedding to adjust text-condition features toward the style reference. On 5000 MS COCO (content) $\times$ WikiArt (style) pairs, the 5-processor StyleKV setting without AdaIN obtains the lowest Art-FID (85.43). Adding AdaIN reduces LPIPS from 0.629 to 0.599, with a modest increase in Art-FID. We release the AdaIN StyleKV project repository [32], including StyleKV Control Studio, a Gradio interface for adjusting γ , Canny thresholds, and AdaIN coefficients.

Keywords: style transfer; diffusion models; self-attention; adaptive instance normalization; training-free; ControlNet

1. Introduction

Artistic style transfer aims to combine the content of one image with the visual style of another. A useful result should preserve the scene structure of the content image while transferring texture, brushwork, and color from the reference style image.

Early work synthesized textures through non-parametric sampling [1]. Gatys et al. [2] later formulated style transfer as optimization over Gram matrices of VGG features. Feedforward networks [3,4] and Adaptive Instance Normalization (AdaIN) [4] made arbitrary style transfer much faster, while GAN-based methods improved image fidelity [5,6,7]. These methods still have limits. GAN-based models often generalize poorly outside the style domains seen during training, and optimization-based methods remain too slow for interactive use. Most early methods also provide little direct control over the balance between style strength and content preservation.

Diffusion models changed the design space for image generation [8,9,10]. Stable Diffusion [11] combines latent-space denoising with cross-attention conditioning, which makes high-resolution synthesis controllable through text prompts. Several diffusion-based style transfer methods have followed [12,13,14,15,16,17]. InST [12] and StyleDiffusion [13] can produce strong results, but they require per-image optimization or textual inversion. Training-free methods such as DiffStyle [14] and Style Injection [16] avoid this cost, but they may alter the content geometry or fail to capture global color statistics. Self-attention is useful for local texture transfer, yet it does not by itself enforce palette consistency. Continuous control over style strength is also uncommon.

AdaIN StyleKV addresses these issues with three inference-time mechanisms inside a frozen SDXL model. The project began with a simple AdaIN prototype using Canny ControlNet and the target-block selection suggested by InstantStyle [17]. Applying AdaIN to one or two UNet blocks improved color consistency, but local brushwork and texture transfer remained weak. We therefore added a Style Injection inspired K/V fusion step. Instead of replacing content K/V features with style K/V features, we blend them with a coefficient $\gamma \in [0, 1]$. The fusion is restricted to selected self-attention processors in `up_blocks.0.attentions.1`, rather than being applied across the whole UNet.

The AdaIN branch handles the color statistics that K/V fusion does not model explicitly. We adapt the cross-modal AdaIN formulation from StyleStudio [31] and apply it at the same target UNet block. A pretrained image encoder, IP-Adapter’s image encoder in our implementation, extracts a style embedding. Its channel-wise mean and variance are then used to modulate text-condition features. Canny ControlNet supplies the spatial anchor through an edge map, while K/V fusion transfers local texture.

We evaluate seven variants on 5000 COCO [18] $\times$ WikiArt [19] pairs. Art-FID [20] measures proximity to the style

domain, FID [21] measures distributional distance from the content domain, and LPIPS [22] measures per-pair perceptual change. The 5-processor StyleKV variant without AdaIN obtains the lowest Art-FID (85.43), but its LPIPS is 0.629. Adding AdaIN increases Art-FID to 90.76 while reducing LPIPS to 0.599, suggesting a more conservative but more stable perceptual change. We also analyze the effects of block count, processor count, γ , and AdaIN, and provide the AdaIN StyleKV project repository with a Gradio interface for interactive parameter control [32].

2. Related Work

2.1. Neural Style Transfer

Gatys et al. [2] introduced neural style transfer by matching Gram matrices of VGG [23] features in an optimization loop. Johnson et al. [24] trained feedforward transformation networks with perceptual losses, and Huang & Belongie [4] proposed AdaIN for arbitrary style transfer in a single forward pass. Later work explored attention mechanisms [25], Transformers [26], reversible flows [27], and dual-branch learning [28]. These methods improved speed and generalization, but highly abstract or non-photorealistic styles can still produce blurred details or flattened textures.

2.2. GAN-Based Style Transfer

CycleGAN [5] used cycle-consistency losses for unpaired image translation, and StarGAN [6] extended this setting to multiple domains in a single model. StyleGAN [7] and later versions [29,30] provide a controllable latent space for high-quality image synthesis. However, StyleGAN is not designed for cross-domain translation. Applying it to arbitrary style transfer usually requires additional fine-tuning or feature disentanglement, and the results can be unstable outside the training distribution.

2.3. Diffusion-Based Style Transfer

Diffusion models now lead in high-quality image generation. Among them, latent diffusion models (LDM) [11] achieve efficient training and inference by compressing images into a low-dimensional latent space. For style transfer, researchers have proposed several diffusion-based methods:

InST [12] uses textual inversion to embed a style image into the text token space, but each new style requires a separate inversion run. StyleDiffusion [13] combines CLIP-based losses with fine-tuning to separate content and style in feature space. DiffStyle [14] operates in the diffusion H-space without training, although direct application to Stable Diffusion may shift semantic layout. Style Injection [16] replaces self-attention K/V features with style-image features, giving a training-free route to texture transfer, but it has no explicit spatial anchor. InstantStyle [17] shows that only a subset of UNet blocks is strongly related to style injection. StyleStudio [31] studies selective control of style elements in text-driven style transfer.

Table 1 compares these methods with AdaIN StyleKV. Our method is training-free, uses a Canny edge condition for structure preservation, applies cross-modal AdaIN for

Table 1. Comparison of our method with representative style transfer methods. TF: Training-Free (no fine-tuning); SP: Structure Preservation (explicit spatial constraints); CT: Color Transfer (global color handling); CI: Controllable Intensity (explicit continuous control).

Method	TF	SP	CT	CI
InST [12]	×	×	✓	×
StyleDiffusion [13]	×	×	✓	✓
DiffStyle [14]	✓	×	×	×
Style Injection [16]	✓	×	×	×
InstantStyle [17]	✓	×	×	×
AdaIN StyleKV (Ours)	✓	✓	✓	✓

color alignment, and exposes a scalar γ for continuous style-intensity control.

2.4. Conditional Control in Diffusion Models

ControlNet [15] adds spatial conditions such as edges, depth, and pose to a frozen diffusion model through zero-convolution layers. We use the Canny variant. It takes an edge map from the content image, processes it through the ControlNet branch, and injects the resulting features into the frozen UNet. The ControlNet weights must match the base model. All experiments use SDXL with `diffusers/controlnet-canny-sdxl-1.0`. An SD1.5 setup would require a matched weight such as `l1lyasviel/control_v11p_sd15_canny`. We use SDXL for its higher native resolution and stronger semantic representation.

3. Method

AdaIN StyleKV uses a frozen SDXL backbone with a Canny ControlNet. At inference time, it combines cross-modal AdaIN for aligning text-condition features with style-image color statistics, selective KV fusion for injecting style K/V features into selected self-attention processors, and Canny edge guidance for preserving scene geometry. The pipeline has five stages.

3.1. Overall Framework

Stage 1: Preprocessing. Content and style images go into the VAE encoder, producing latents z_{content} and z_{style} . A Canny edge detector (thresholds adjustable in our frontend) extracts a contour map from the content image as the ControlNet spatial condition c_{canny} .

Stage 2: DDIM Inversion. We run DDIM inversion on both latents separately to obtain noise trajectories $\{z_t^{\text{content}}\}$ and $\{z_t^{\text{style}}\}$ across all T timesteps. These trajectories provide the timestep-aligned content and style latents used for AdaIN modulation and KV caching.

Stage 3: Cross-Modal AdaIN. At each denoising timestep, before the UNet forward pass, we modify the text-condition embeddings at the target block(s). AdaIN uses channel statistics from a pretrained style-image embedding, extracted with the IP-Adapter image encoder in our implementation. No additional adapter parameters are trained.

Stage 4: KV Fusion. The target block is `up_blocks.0.attentions.1`. During the style image forward pass at timestep t , we record $\{K_{\text{style}}, V_{\text{style}}\}$ from selected self-attention processors in that block. During content denoising at the same timestep, each selected processor uses:

$$\begin{aligned} K_{\text{fused}} &= (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}}, \\ V_{\text{fused}} &= (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}}, \end{aligned} \quad \gamma \in [0, 1].$$

Non-target processors run standard self-attention with no caching and no blending. The content query Q_{content} stays untouched throughout.

Stage 5: Decoding. After T denoising steps, z_0 goes through the VAE decoder to produce the stylized output.

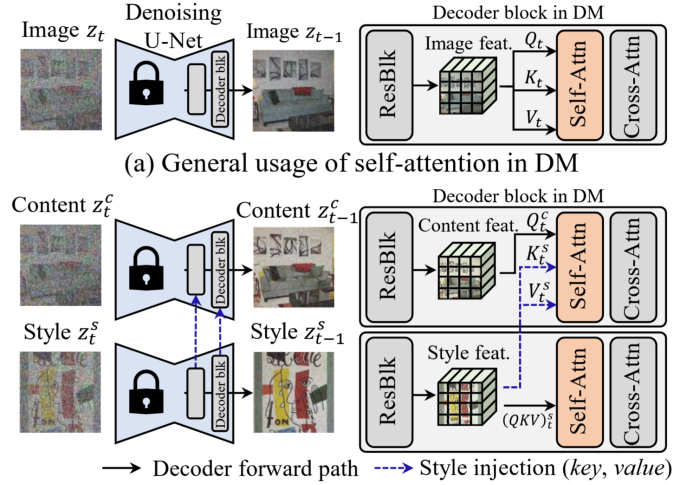


Fig. 2. Overall pipeline of AdaIN StyleKV. Content and style images enter the VAE in parallel; a Canny edge map of the content anchors ControlNet. DDIM inversion produces latent trajectories for both images. Cross-modal AdaIN fuses text and image conditions at the target UNet block, and cached style K/V are blended with content K/V inside selected self-attention processors. The VAE decoder produces the final output.

Fig. 2 gives the visual summary of this five-stage flow.

3.2. Selective KV Fusion

3.2.1 Self-Attention Recap

The UNet self-attention layer projects a feature map X into query, key, and value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

then computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

In standard generation, Q , K , and V come from the same image. In our setting, Q remains content-derived, while K and V are blended with style-derived features. This keeps the attention query tied to the content layout while allowing the retrieved features to carry texture and appearance from the style image.

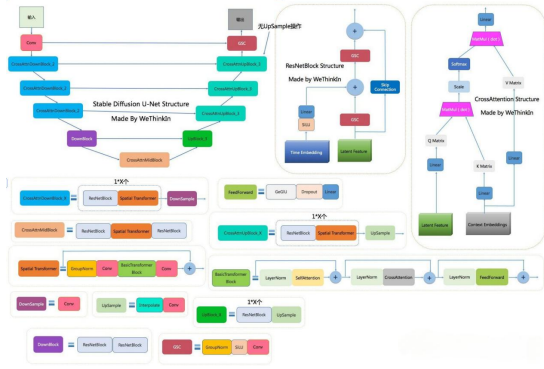


Fig. 3. U-Net architecture in Stable Diffusion, showing the encoder, middle block, and decoder with their ResNet and self-attention layers. We confine KV fusion to the target block `up_blocks.0.attentions.1` (red), leaving all other attention processors untouched.

3.2.2 Weighted KV Fusion

Following Style Injection [16], we operate on self-attention processors inside `up_blocks.0.attentions.1`. Unlike hard replacement, we use a linear blend:

$$K_{\text{fused}} = (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}} \quad (3)$$

$$V_{\text{fused}} = (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}} \quad (4)$$

where $\gamma \in [0, 1]$ controls style strength. When $\gamma = 0$, the processor uses the content K/V. When $\gamma = 1$, it uses the style K/V. Intermediate values interpolate between the two. This operation requires no gradient updates and is less abrupt than direct K/V replacement.

3.2.3 Target Block and Processor Selection

InstantStyle [17] shows that only some UNet blocks are strongly related to style injection. We therefore fix a single decoder block, `up_blocks.0.attentions.1`, as the StyleKV injection site. All StyleKV experiments in this paper operate only in that block. We vary the number of participating self-attention processors:

- **1 proc:** minimal StyleKV injection
- **5 procs:** default configuration
- **10 procs:** stronger StyleKV injection with higher risk of detail loss

The 5-processor configuration gives the best balance between style transfer and content preservation in our experiments, so we use it as the default unless stated otherwise.

3.3. Cross-Modal AdaIN for Text and Image Conditions

KV fusion mainly affects local texture and brushwork. It does not reliably capture the global palette of the style image. Following the cross-modal AdaIN idea in StyleStudio [31],

Table 2. Target block configurations in the early naive AdaIN prototype. 1blk/2blk indicate the number of UNet target blocks where AdaIN is applied; in contrast, StyleKV experiments are fixed at `up_blocks.0.attentions.1` and vary the number of self-attention processors.

Config	AdaIN Target UNet Block(s)	Variant
1blk	<code>up0.attn1</code>	V1: <code>adain_1blk</code>
2blk	<code>up0.attn1; down2.attn1</code>	V2: <code>adain_2blk</code>

we use image-feature statistics to modulate text-condition features and improve color consistency.

A frozen image encoder, the IP-Adapter image encoder in our implementation, produces a style embedding f_{style} . At the same target UNet block used for KV fusion, we take the existing text-condition features f_{text} and re-normalize them:

$$f_{\text{adain}} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \mu(f_{\text{style}}) \quad (5)$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ are computed per channel. The implementation exposes two coefficients, α and β :

$$f_{\text{adain}}^{(\beta)} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \beta \mu(f_{\text{style}}) \quad (6)$$

$$f_{\text{out}} = (1 - \alpha)f_{\text{text}} + \alpha f_{\text{adain}}^{(\beta)} \quad (7)$$

With the default setting $\alpha=1.0, \beta=1.0$, this reduces to Eq. 5. The operation keeps the semantic direction of the text embedding while matching its channel statistics to the style embedding. This changes the color and atmosphere implied by the condition without introducing new trainable modules.

3.4. ControlNet Integration

Canny ControlNet provides the spatial constraint. It receives the content image edge map and injects the resulting features into the frozen UNet through the ControlNet branch. We use the SDXL-matched weights `diffusers/controlnet-canny-sdxl-1.0` without further training.

3.5. Inference

Everything above happens at inference time inside a completely frozen ϵ_{θ} . At timestep t , with text prompt c and Canny condition c_{canny} , the noise prediction is:

$$\hat{\epsilon} = \epsilon_{\theta}(z_t, t, c, c_{\text{canny}}) \quad (8)$$

The AdaIN modulation and KV fusion are injected as side-effects during the forward pass through the target block; the model weights never move. The output follows the five-stage pipeline described earlier.

4. Experiments and Results

4.1. Experimental Setup

4.1.1 Datasets

Content dataset: We use 5000 content images from the MS COCO 2017 dataset [18], each accompanied by a corresponding text prompt to assist the diffusion model in understanding the image content.

Style dataset: We use 5000 style images from the WikiArt dataset [19], covering various artistic styles such as Impressionism, Cubism, and Abstract art.

Image pairing: 5000 content images and 5000 style images are paired one-to-one in sequential order, yielding 5000 pairs in total.

Data scale: A total of 5000 stylized images are generated for quantitative evaluation.

4.1.2 Evaluation Metrics

We track three numbers, all in the lower-is-better direction, but they capture different things. **Art-FID** [20] measures distributional distance from the target style set; it is our primary gauge of how well the style came through. **FID** [21] does the same against the content set, giving a rough proxy for content-domain proximity (though it’s too global to serve as a standalone structure-preservation score). **LPIPS** [22] computes per-pair perceptual distance between each output and its content image. A low LPIPS means the image “reads” similar to the original; a high LPIPS means the transfer visibly shifted things. Since any interesting stylization will push LPIPS up somewhat, we read it alongside Art-FID and FID, not in isolation. No single metric tells the full story; the three-way trade-off is the point.

4.1.3 Hardware Configuration and Hyperparameters

Experiments were conducted on the AutoDL platform using a rented NVIDIA V100-32GB GPU. Hardware and hyperparameter configurations are shown in Table 3 and Table 4, respectively.

Table 3. Hardware configuration and environment.

Configuration Item	Parameter
GPU	NVIDIA V100 32GB × 1
Acceleration Library	xformers
Sampling Steps	50 steps (DDIM)
Resolution	512 × 512 (center crop)
Total Generated Images	5000

Table 4. Hyperparameter settings. $\gamma = 0.6$ is the default trade-off point selected from the sensitivity analysis in Section 4.4; KV fusion defaults to 5 self-attention processors in `up_blocks.0.attentions.1`; AdaIN fusion intensity defaults to 1.0.

Parameter	Value	Description
γ	0.6	Style fusion intensity
<code>up0.attn1_procs</code>	5	KV fusion processor count
<code>adain_alpha</code>	1.0	AdaIN fusion intensity
<code>adain_beta</code>	1.0	AdaIN fusion bias
CFG scale	7.5	Classifier-free guidance
DDIM steps	50	Diffusion sampling steps

4.2. Method Variant Design

We evaluate seven configurations (Table 5). V1 to V5 correspond to the early AdaIN-only prototype: V1 and V2 use AdaIN at one or two target blocks, V3 and V4 remove AdaIN under the same block settings, and V5 removes ControlNet while keeping AdaIN. V6 and V7 add 5-processor StyleKV fusion. V6 disables AdaIN, and V7 is the full method. We use the term “ablation” only for single-factor comparisons: V6 vs. V7 isolates AdaIN, and V1 vs. V5 isolates ControlNet. The 1blk/2blk and 1/5/10-proc settings are configuration sweeps or sensitivity analyses.

Table 5. Design of 7 method variants. CtrlN = ControlNet; Scope = target blocks (blk) or processors (p); KV = KV fusion enabled.

#	Variant	CtrlN	Scope	AdaIN	γ	KV
V1	adain_1blk	✓	1blk	✓	N/A	×
V2	adain_2blk	✓	2blk	✓	N/A	×
V3	noadain_1blk	✓	1blk	×	N/A	×
V4	noadain_2blk	✓	2blk	×	N/A	×
V5	adain_no_ctrl	×	1blk	✓	N/A	×
V6	proc05_no_adain	✓	5p	×	0.6	✓
V7	proc05_adain	✓	5p	✓	0.6	✓

Fig. 4 and Fig. 5 show visual comparisons across different content-style pairings, focusing on style injection, color coordination, and content preservation.

Quantitative Comparison Across Evaluation Metrics

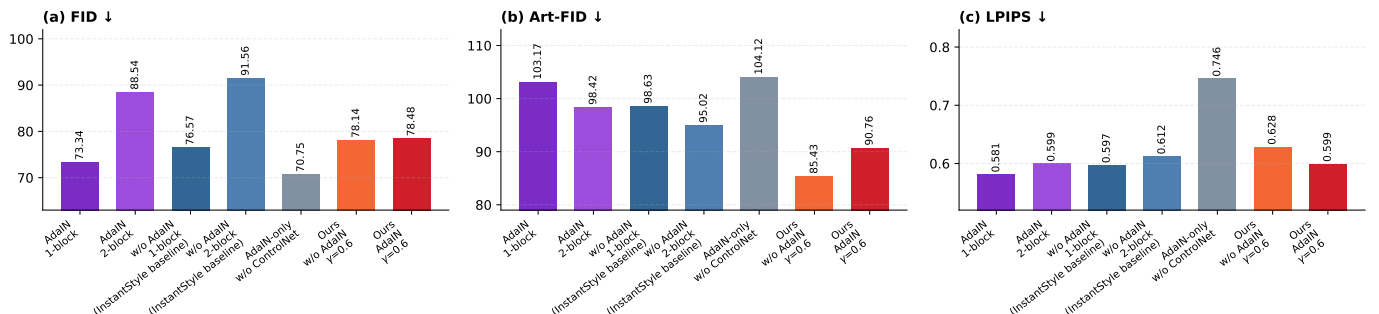


Fig. 6. Quantitative comparison of 7 method variants across FID, Art-FID, and LPIPS metrics (lower is better for all three). FID reflects distribution distance from the content domain, Art-FID reflects proximity to the target artistic style domain, and LPIPS reflects the perceptual-level image change magnitude. V6 (Ours w/o AdaIN) achieves the lowest Art-FID, indicating stronger style injection; compared with V6, the full method V7 (Ours AdaIN, $\gamma = 0.6$) reduces LPIPS, indicating milder perceptual change.



Fig. 4. Horizontal comparison of outputs from different method configurations on the “man” content image. The upper labels indicate the corresponding method configuration and allow comparison of style injection intensity and content preservation.

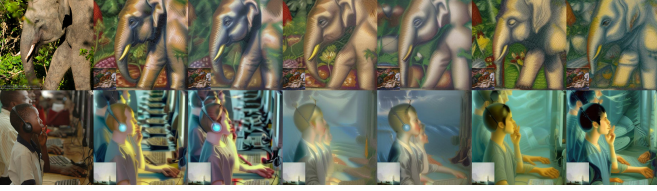


Fig. 5. Method variant comparison on two different content-style pairings.

4.3. Quantitative Results

Table 6 presents the quantitative evaluation results of the 7 variants on 5000 generated images.

Table 6. Quantitative evaluation results of 7 variants on 5000 generated images. FID measures distribution distance from the content domain (lower indicates closer overall feature distribution to original content); Art-FID measures distribution distance from the style domain (lower indicates higher style similarity); LPIPS measures the degree of perceptual change (stylization inevitably introduces perceptual changes; moderate LPIPS values indicate effective style transfer while content remains recognizable). Bold indicates the full method (V7).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (adaIn_1blk)	73.34	103.17	0.581
V2 (adaIn_2blk)	88.54	98.42	0.599
V3 (no_adaIn_1blk)	76.57	98.63	0.597
V4 (no_adaIn_2blk)	91.56	95.02	0.612
V5 (adaIn_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adaIn)	78.14	85.43	0.629
V7 (proc05_adaIn)	78.48	90.76	0.599

Analysis. V6, which uses 5 StyleKV processors without AdaIn, obtains the best Art-FID in the sweep (85.43). This indicates stronger style-domain alignment, but LPIPS increases to 0.629, meaning the output also moves further from the content image. Adding AdaIn in V7 increases Art-FID to 90.76 but reduces LPIPS to 0.599. This suggests that AdaIn weakens the most aggressive texture replacement while improving color alignment.

The AdaIn-only baselines (V1, V2) improve color consistency but do not reduce Art-FID below 98. This supports the role of KV fusion in local texture transfer. V5 removes ControlNet and obtains the lowest FID (70.75), but its LPIPS rises to 0.746. Without the Canny constraint, the generated images remain close to the content domain in aggregate while losing more local structure. Increasing the number of AdaIn target blocks (V1→V2, V3→V4) strengthens condition modulation, but it also increases content change.

Fig. 7 converts the three metrics to ranks, where 1 is best. No variant is best on all metrics. V5, which removes

ControlNet, ranks first on FID but worse on Art-FID and LPIPS. V6 ranks first on Art-FID but worse on LPIPS. V7 gives a more conservative trade-off: it is not the best on Art-FID, but it improves LPIPS relative to V6.

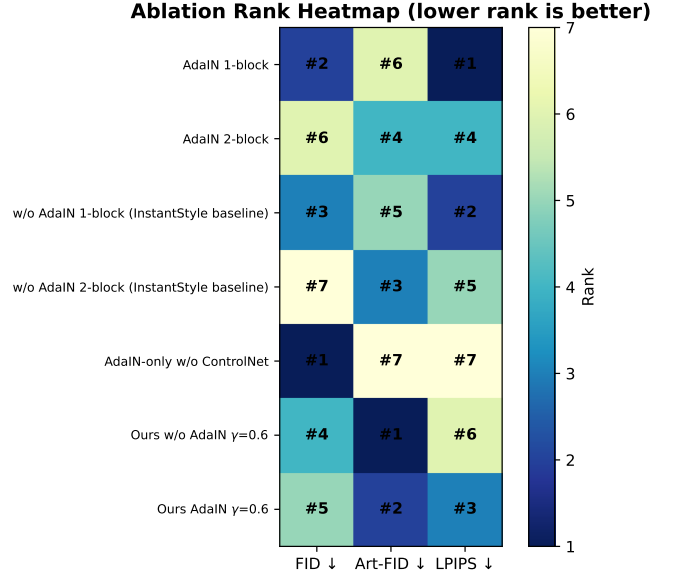


Fig. 7. Ranking heatmap of the 7 method variants across FID, Art-FID, and LPIPS metrics (lower rank indicates better performance).

Fig. 4 and Fig. 8 display visual comparison results of the variants.

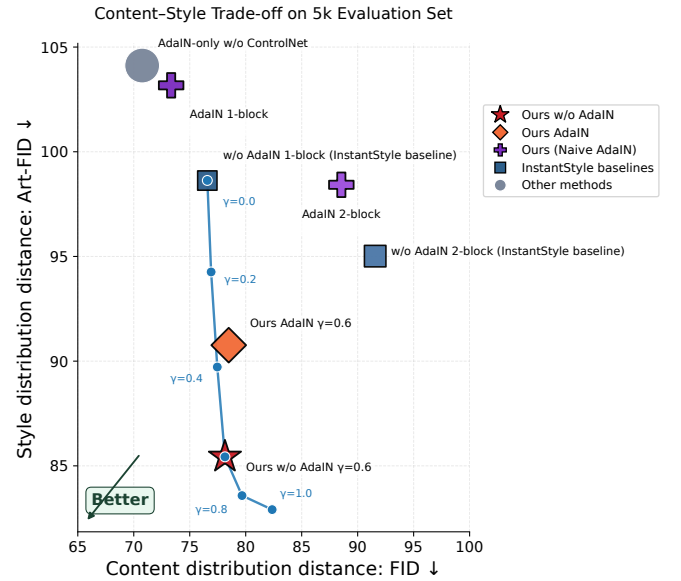


Fig. 8. Content-style trade-off comparison of method variants on FID and Art-FID.

4.4. Gamma Intensity Coefficient Sensitivity Analysis

To analyze the impact of the style fusion intensity coefficient γ on generation results, we fix the configuration of 5 self-attention processors in `up_blocks.0.attentions.1` with AdaIn and ControlNet, and vary only the value of γ , testing $\gamma \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. Experimental results are shown in Table 7.

Table 7. Results of γ intensity coefficient sensitivity analysis (5-processor configuration in `up_blocks.0.attentions.1`, fixed AdaIN + ControlNet). $\gamma = 0$ mainly retains content features with weak style injection; $\gamma = 1$ yields the highest stylization degree but with more noticeable content change. Bold indicates our default configuration, not necessarily the single-metric optimum.

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697

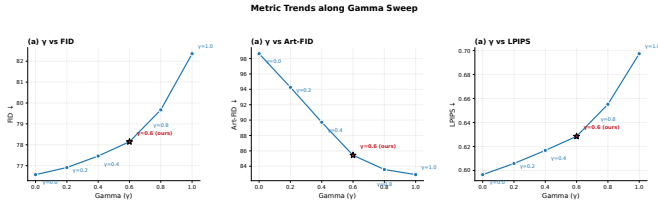


Fig. 9. Impact of γ on FID, Art-FID, and LPIPS. As γ increases, style fidelity generally improves but content change also increases; $\gamma = 0.6$ is our adopted default trade-off point.

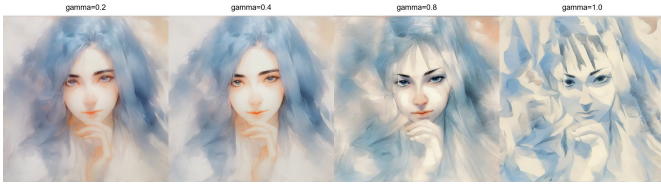


Fig. 10. Style transfer results at different γ values under the AdaIN-enabled 5-processor configuration in `up_blocks.0.attentions.1` (our method). From left to right: $\gamma = 0.2, 0.4, 0.8, 1.0$. Style texture becomes stronger as γ increases.

Conclusion: As γ increases, style texture and color expression become stronger, and generated results move closer to the target artistic style distribution. Content fidelity and fine-grained structure also become more vulnerable to degradation. Smaller γ values favor content preservation, while larger values favor style injection. Combining quantitative metrics with visual results in Fig. 9 and Fig. 10, we select $\gamma = 0.6$ as the default trade-off point that leans toward content stability.

4.5. Qualitative Analysis of StyleKV Target Processor Count

Table 2 has already explained that `1blk/2blk` belong to target block configurations in the early naive AdaIN prototype, whereas experiments prefixed with `proc` concern the number of processors within StyleKV. These are not the same ablation dimension, so this section does not compare them as a unified quantitative ablation group. To observe the impact of the number of processors within StyleKV on style injection intensity and content preservation, we fix the target block to `up_blocks.0.attentions.1` and vary only the number of self-attention processors participating in K/V fusion within that block for qualitative visual analysis.

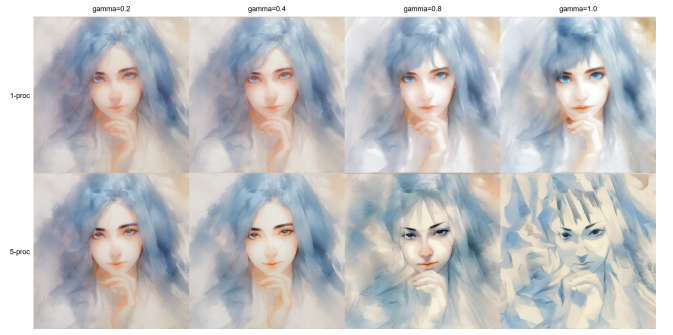


Fig. 11. Comparison of style transfer results under AdaIN-enabled 1-processor and 5-processor configurations in `up_blocks.0.attentions.1` at different γ values. Top row: 1-proc StyleKV configuration; bottom row: 5-proc StyleKV configuration (our method). More target processors yield stronger style injection (more pronounced style texture in the bottom row) while still maintaining good content structure in the shown examples.

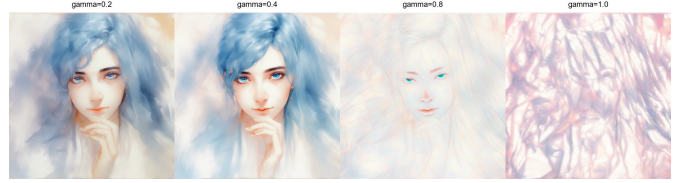


Fig. 12. Style transfer results under the AdaIN-enabled 10-processor configuration in `up_blocks.0.attentions.1` at different γ values. Compared with the 5-proc configuration in the same block, 10-proc yields stronger style injection at the same γ , but content detail preservation is slightly inferior to the 5-proc configuration.

Observation: Fig. 11 shows that, within the same target block, the 5-proc configuration produces stronger texture transfer than the 1-proc configuration. Fig. 12 shows that the 10-proc configuration further increases stylization, but it is more likely to degrade facial details and local structure at high γ . We therefore use 5 processors in `up_blocks.0.attentions.1` as the default StyleKV configuration.

4.6. Qualitative Result Analysis

4.6.1 Comparison of Different Block/Processor Configurations and AdaIN Effects

Table 8 compares the generation results of four naive AdaIN baseline configurations. Fig. 13 further illustrates the impact of the AdaIN module on color coordination under the 1-block configuration.

Table 8. Qualitative comparison of different target block counts and AdaIN configurations. Configurations using AdaIN exhibit better color coordination and more uniform style coverage; configurations without AdaIN show stronger style transfer but more noticeable content structure degradation.

Variant	#Blocks	Observation
adain_1blk	1	Concentrated style injection, noticeable texture transfer
adain_2blk	2	Uniform style coverage, better color coordination
no_adain_1blk	1	Strong style transfer, slightly inferior content preservation
no_adain_2blk	2	Enhanced style injection, more blurred structure

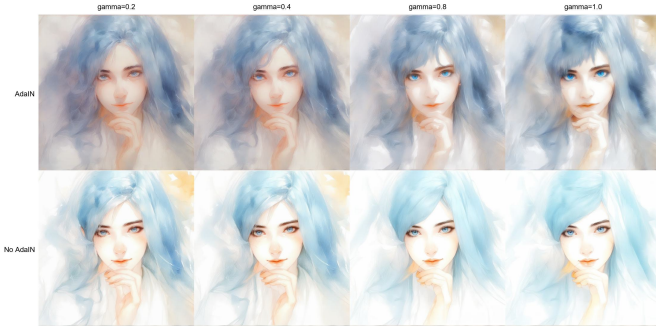


Fig. 13. Impact of the AdaIN module on style transfer (1-block configuration). Top row: results with AdaIN; bottom row: results without AdaIN. The AdaIN module improves color coordination and overall atmospheric consistency, with the difference being more pronounced at higher γ values.

Fig. 1 (the teaser at the beginning of the paper) presents the overall transfer results of our method in a comprehensive comparison.

4.6.2 Impact of Prompts

In qualitative experiments, we observe that prompt quality has a clear effect on generation results. When using simple prompts, the model has limited content understanding and lacks detail; when using detailed prompts, the model can more accurately capture user intent, and style consistency and visual quality are generally better. This suggests that in style transfer tasks, text prompts are an important factor in guiding the diffusion model to understand content semantics.

4.7. Key Experimental Findings

4.7.1 Trade-off Between Canny Edge Map Resolution and Inference Time

The granularity of the Canny edge map directly affects generation quality and inference efficiency. Table 9 tests inference times at different Canny resolutions.

Finer Canny edge maps improve generation quality because the model receives more structural information. Excessively fine edge maps can also make the model over-respond to local edges, producing outlining artifacts, fragmented textures, or unnatural local details. Inference time increases markedly with Canny resolution, so the method

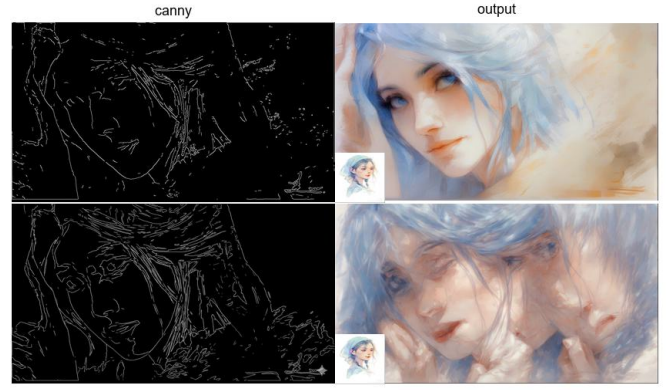


Fig. 14. Comparison of structural conditions and generation results at different Canny resolutions. Left column: Canny edge maps; right column: corresponding stylized outputs, with the reference style image embedded in the bottom-left corner of each output.

Table 9. Canny resolution vs. inference time. Higher resolution provides richer structure but increases latency and may introduce artifacts.

Resolution	Canny Size	Time
1080×599	same as input	~12 s
5774×1536	same as input	~68 s

needs a practical balance between detail and cost. Fig. 14 shows the difference in Canny edge granularity across resolutions. The frontend therefore lets users adjust Canny edge-detection parameters.

4.7.2 Device Resource Constraints

Diffusion models still pose certain challenges for consumer-grade GPUs. In our setup, the Stable Diffusion base model uses about 7 to 8 GB of VRAM; adding VAE, ControlNet, and feature caching brings the total to about 15 to 20 GB. Even with a V100-32 GB, batch generation faces VRAM pressure and GPU resource contention. Compared with injection across more blocks or processors, our approach uses selective processor injection within `up_blocks.0.attentions.1` (5 self-attention processors by default), which controls the extra computation and caching overhead.

5. Interactive Frontend

For practical use, we built a Gradio frontend called **StyleKV Control Studio**.

5.1. Frontend Functional Modules

(1) Image Input Module: content image upload, style image upload, automatic generation and preview of the Canny edge map.

(2) Prompt Control Module: positive prompt (describing the desired generation result), negative prompt (elements to avoid), with support for long text input.

(3) ControlNet/Canny Parameter Adjustment: Canny Low/High Threshold (edge detection sensitivity), Conditioning Scale (ControlNet control strength), Bilateral Filter parameters (edge preprocessing denoising).

(4) **AdaIN Style Fusion Module:** AdaIN Alpha (fusion intensity), AdaIN Beta (fusion bias), with real-time preview support.

(5) **Generation Parameter Control:** Steps (DDIM sampling steps), CFG Scale (classifier-free guidance), Seed (random seed), KV fusion intensity γ .

(6) **Output Management:** output directory setting, real-time generation status display, result gallery display.

5.2. Frontend Features and Value

The frontend supports parameter updates without restarting, Canny edge-map preview, JSON configuration export, and batch generation. Users can inspect how each parameter affects the output through sliders, switches, and preview images. Fig. 15 shows the interface layout.

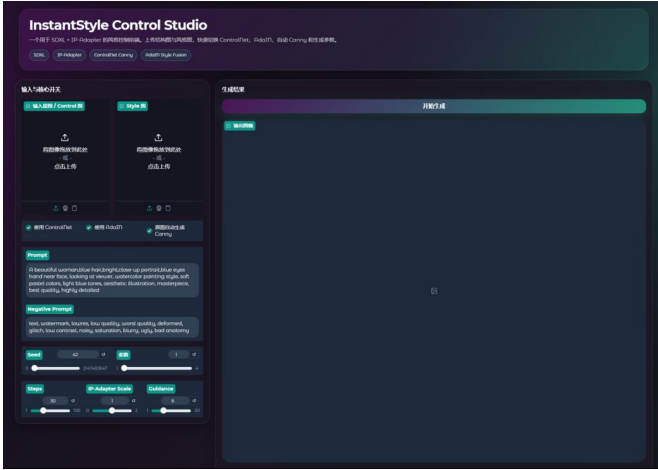


Fig. 15. Interface of the StyleKV Control Studio interactive frontend. Through the left panel, users can upload content and style images, adjust ControlNet/Canny parameters, set AdaIN fusion coefficients, and configure generation parameters; the right panel displays real-time Canny edge map previews and stylized output results.

6. Future Work

6.1. Quantitative Comparison with Existing Methods

Our current experiments focus on self-ablation to verify the effectiveness of each component and have not yet been compared with methods such as InST [12], StyleDiffusion [13], and InstantStyle [17] under equivalent conditions. We plan to conduct systematic inter-method comparisons under a unified dataset and evaluation protocol.

6.2. Adaptive Canny Edge Map Generation

Currently, Canny thresholds must be manually adjusted, and optimal values vary depending on image content. We plan to introduce better edge extraction models such as HED and RINDNet to realize adaptive edge map generation, reducing the need for manual parameter tuning.

6.3. VLM-Based Automatic Prompt Generation

Currently, prompts need to be manually written or generated with the help of external models. We plan to introduce open-

source vision-language models (e.g., Qwen-VLM, LLaVA) to automatically generate high-quality prompts for style images, further lowering the usage barrier.

6.4. Cross-Architecture Adaptation and Lightweighting

Our method is validated on SDXL, and its effective attention processor selection and fusion strategy is architecture-dependent. We plan to adapt the KV fusion mechanism to other diffusion architectures such as SD1.5 and SD3, re-identify the effective self-attention processors under each architecture, and explore lightweighting schemes such as distillation and quantization to reduce dependence on high-VRAM devices.

6.5. Incorporating Tile ControlNet

High-resolution Canny edge maps may introduce artifacts. We plan to use Tile ControlNet to mitigate artifact issues at high resolution through patch-wise processing while maintaining detail richness.

6.6. Processor-Level Contribution Analysis

Currently, we only test naive AdaIN configurations with 1 or 2 target blocks, and StyleKV configurations with 1, 5, or 10 self-attention processors in `up_blocks.0.attentions.1`. We plan to individually test the style transfer contribution of different self-attention processors in U-Net, establishing a processor-to-style mapping to provide a theoretical basis for method optimization.

6.7. Multi-Condition ControlNet Fusion

The current system only integrates Canny ControlNet. Future work will test multiple ControlNets, such as edge, depth, and pose, for stronger content preservation.

7. Conclusion and Limitations

7.1. Conclusion

We put forward AdaIN StyleKV, a training-free style transfer pipeline that combines three lightweight mechanisms inside a frozen SDXL+ControlNet backbone: (1) selective KV fusion, which caches style K/V from a handful of self-attention processors in `up_blocks.0.attentions.1` and blends them with content K/V via a single γ ; (2) cross-modal AdaIN, adapted from StyleStudio [31], to pull text-condition features toward the style image’s color statistics; and (3) Canny ControlNet for spatial anchoring. On 5000 COCO×WikiArt pairs, the full configuration lands at Art-FID 90.76 with LPIPS 0.599, while the AdaIN-free StyleKV variant pushes Art-FID down to 85.43 at the cost of higher perceptual disruption (LPIPS 0.629). We also release a Gradio frontend that exposes every parameter to the user in real time.

7.2. Limitations

Although our method produces competitive results, the following limitations remain, as listed in Table 10. These

limitations also constitute the key focus areas of the future work outlined in Section 6.

Table 10. Major limitations of our method and corresponding improvement directions.

Limitation	Description	Improvement
No external SOTA comparison	Only self-variant ablation; not compared with InST, StyleDiffusion, InstantStyle, etc.	Reproduce and compare under unified protocol
Insufficient processor ablation	Only tested few processor configs in <code>up0.attn1</code>	Full processor-level contribution analysis
Single ControlNet	Only Canny tested; other modalities not explored	Multi-condition ControlNet fusion
External prompt dependency	Prompts from external models; no integrated VLM	Integrate Qwen-VLM / LLaVA
Limited evaluation metrics	FID, Art-FID, LPIPS only; no perceptual/semantic coverage	Add metrics + human subjective study
High device requirements	Needs V100-class GPU; not consumer-friendly	Distillation & quantization

7.3. Future Outlook

Style transfer is far from solved. As diffusion models keep evolving, a few directions strike us as especially worth pursuing: pushing training-free methods toward finer style disentanglement (so users can independently control brush scale, palette, and texture density); fusing multiple conditions (text, reference image, color palette) without them fighting each other; and mapping out which UNet layers actually matter for style, so future methods can be leaner by design. Getting these pipelines to run on consumer hardware, through distillation or quantization, would also go a long way toward making them useful outside the lab.

Acknowledgments

We are grateful to the AutoDL platform for GPU resources and to the open-source community behind Stable Diffusion, ControlNet, and the diffusers library. Without these foundations this project would not have been possible.

References

- [1] A. A. Efros and T. K. Leung, “Texture synthesis by non-parametric sampling,” in *Proc. ICCV*, 1999.
- [2] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image style transfer using convolutional neural networks,” in *Proc. CVPR*, 2016.
- [3] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky, “Texture networks: Feed-forward synthesis of textures and stylized images,” in *Proc. ICML*, 2016.
- [4] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *Proc. ICCV*, 2017.
- [5] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Proc. ICCV*, 2017.
- [6] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *Proc. CVPR*, 2018.
- [7] T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proc. CVPR*, 2019.
- [8] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proc. ICML*, 2015.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.
- [10] Y. Song and S. Ermon, “Improved techniques for training score-based generative models,” in *Proc. NeurIPS*, 2020.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. CVPR*, 2022.
- [12] Y. Zhang, N. Huang, F. Tang, H. Huang, C. Ma, W. Dong, and C. Xu, “InST: Inversion-based style transfer with diffusion models,” in *Proc. CVPR*, 2023.
- [13] Z. Wang, L. Zhao, W. Xing, and D. Lu, “StyleDiffusion: Controllable disentangled style transfer via diffusion models,” in *Proc. ICCV*, 2023.
- [14] J. Jeong, M. Kwon, and Y. Uh, “DiffStyle: Training-free style transfer with diffusion models,” *arXiv preprint arXiv:2312.01714*, 2023.
- [15] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proc. ICCV*, 2023.
- [16] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Style Injection: Training-free style transfer using self-attention,” *arXiv preprint arXiv:2312.08008*, 2023.
- [17] H. Wang, Q. Wang, X. Bai, Z. Qin, and A. Chen, “InstantStyle: Free lunch towards style-preserving in text-to-image generation,” *arXiv preprint arXiv:2404.02733*, 2024.
- [18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014.

- [19] WikiArt, “WikiArt Visual Art Encyclopedia.” [Online]. Available: <https://www.wikiart.org/>.
- [20] M. Wright and B. Ommer, “ArtFID: Quantitative evaluation of neural style transfer,” in *Proc. GCPR*, 2022, arXiv:2207.12280.
- [21] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Proc. NeurIPS*, 2017.
- [22] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. CVPR*, 2018.
- [23] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015.
- [24] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *Proc. ECCV*, 2016.
- [25] S. Liu, T. Lin, D. He, F. Li, M. Wang, X. Li, Z. Sun, Q. Li, and E. Ding, “AdaAttN: Revisit attention mechanism in arbitrary neural style transfer,” in *Proc. ICCV*, 2021.
- [26] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, “StyTr²: Image style transfer with transformers,” in *Proc. CVPR*, 2022.
- [27] J. An, S. Huang, Y. Song, D. Dou, W. Liu, and J. Luo, “ArtFlow: Unbiased image style transfer via reversible neural flows,” in *Proc. CVPR*, 2021.
- [28] H. Chen, L. Zhao, Z. Wang, H. Zhang, Z. Zuo, A. Li, W. Xing, and D. Lu, “DualAST: Dual style-learning networks for artistic style transfer,” in *Proc. CVPR*, 2021.
- [29] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of StyleGAN,” in *Proc. CVPR*, 2020.
- [30] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” in *Proc. NeurIPS*, 2021.
- [31] M. Lei, X. Song, B. Zhu, H. Wang, and C. Zhang, “StyleStudio: Text-driven style transfer with selective control of style elements,” in *Proc. CVPR*, 2025.
- [32] Sabrinayuzhibo, “AdaIN_StyleKV,” GitHub repository. [Online]. Available: https://github.com/Sabrinayuzhibo/AdaIN_StyleKV (accessed Jun. 14, 2026).

Appendix

A. Complete Data of the 7 Variants

Table 11 lists the complete experimental data of the 7 variants, consistent with Table 5 in the main text.

Table 11. Complete quantitative data of the 7 variants (consistent with Table 5 in the main text).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (adain_1blk)	73.34	103.17	0.581
V2 (adain_2blk)	88.54	98.42	0.599
V3 (no_adain_1blk)	76.57	98.63	0.597
V4 (no_adain_2blk)	91.56	95.02	0.612
V5 (adain_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adain)	78.14	85.43	0.629
V7 (proc05_adain)	78.48	90.76	0.599

B. Complete Gamma Ablation Experiment Data

Table 12 lists the complete data of the γ ablation experiment.

Table 12. Complete γ ablation experiment data (5-processor configuration in `up_blocks.0.attentions.1` + AdaIN + ControlNet, consistent with Table 6 in the main text).

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697